

Master ELISE

Digital Communications 1

LAB 2

Module: Digital Communications 1

Students: Lydia Ali Ahmed & Dalina Chabane

Date: 31 May 2026

Contents

1	Introduction	2
2	Question 1 — IQ Signal Generation	2
2.1	Generation Script	2
2.2	Testing the Flowgraph	3
2.3	Description of the Flowgraph Structure	4
3	Question 2 — IQ Interleaving	4
4	Question 3 — QPSK Constellation Encoding	5
4.1	16-bit to 2-bit Decomposition and Reconstruction	5
4.2	Signal and Constellation Representations	6
4.3	Stereo Audio Playback	7
5	Question 4 — $\pi/4$-DQPSK Modulation	7
5.1	Theoretical Principle	7
5.2	Constellation Visualisation	8
5.3	Fixed Sequence and Frame Synchronisation	10
5.4	Noisy Channel	11
5.5	Phase Offset and Delay Compensation	11
5.6	DQPSK and QPSK Recovery	12
6	Question 5 — Complete Audio Transmission Chain	14
7	Question 6 — Noise Limit and Phase Ambiguity	15
7.1	Noise Tolerance Limit	15
7.2	Phase Ambiguity Immunity	16
8	Question 7 — OFDM with 64 Subcarriers	16
9	Conclusion	17

1 Introduction

The goal of this lab is to implement a physical layer transmission chain for broadcasting a course programme over radio waves, using Orthogonal Frequency Division Multiplexing (OFDM). Following the same approach as digital radio broadcasting (DAB), $K = 16$ channels are made available to transmit audio frames and PDF chapters. Source and channel coding are not covered here and will be addressed in semester 9. The chain transmits 16-bit words representing audio signal samples, at a sampling rate of $F_s = 2.048$ MHz. The entire chain is built and validated step by step in GNU Radio Companion.

2 Question 1 — IQ Signal Generation

2.1 Generation Script

To facilitate validation of the generation chain, sixteen stereo WAV files are generated in Python. Each file contains a sinusoidal signal in phase quadrature: the left channel carries the in-phase component $I(t) = A \cos(2\pi t/T_k)$ and the right channel carries the quadrature component $Q(t) = A \sin(2\pi t/T_k)$, forming the complex baseband signal $s(t) = Ae^{j2\pi t/T_k}$.

The parameters are as follows: the sampling frequency is $F_s = 2.048$ MHz, the period of file k is $T_k = 2^9/k$ milliseconds for k from 1 to 16, and the amplitude spans the full 16-bit signed integer range from -2^{15} to $2^{15} - 1$.

```

1 import numpy as np
2 import wave
3
4 Fs = 2_048_000      # Sampling frequency in Hz
5 K = 16              # Number of channels
6 AMP = 2**15 - 1     # Maximum amplitude (32767)
7
8 for k in range(1, K + 1):
9     T_s = (512.0 / k) / 1000.0    # Period in seconds
10    N = round(T_s * Fs)            # Number of samples per period
11    t = np.arange(N) / Fs          # Time axis
12
13    I = (AMP * np.cos(2 * np.pi * t / T_s)).astype(np.int16)
14    Q = (AMP * np.sin(2 * np.pi * t / T_s)).astype(np.int16)
15
16    # Stereo interleaving: left = I, right = Q
17    stereo = np.empty(2 * N, dtype=np.int16)
18    stereo[0::2] = I
19    stereo[1::2] = Q
20
21    with wave.open(f"sine_{k}.wav", 'w') as wf:
22        wf.setnchannels(2)
23        wf.setsampwidth(2)
24        wf.setframerate(Fs)
25        wf.writeframes(stereo.tobytes())

```

Listing 1: Python script for generating the sixteen stereo WAV files

For $k = 16$, the period is 32 ms and 65,536 samples are generated. For $k = 1$, the period is 512 ms and 1,048,576 samples are generated.

2.2 Testing the Flowgraph

The first flowgraph reads the generated stereo WAV file and displays both I and Q on a time sink. The number of displayed points is adapted to show exactly one complete period:

$$N_{\text{pts}} = (65536 + 500) \times (17 - k)$$

For $k = 16$ this gives 66,036 points, corresponding to a display window of 32.24 ms. The margin of 500 ensures the full period is always visible. The value of the period is verified visually on the time axis.

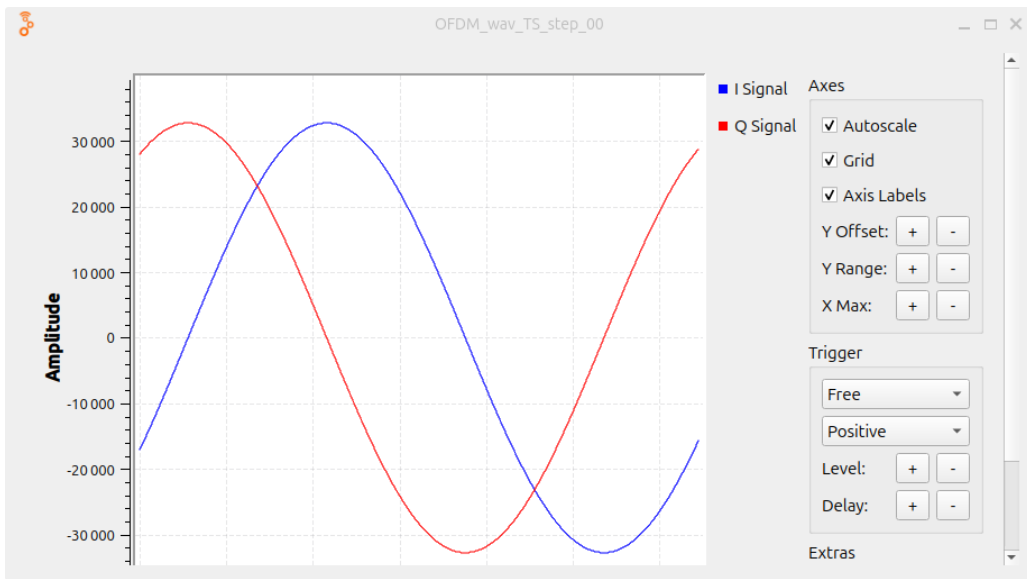


Figure 1: Flowgraph running with the file for $k = 16$: one complete IQ period, $T = 32$ ms

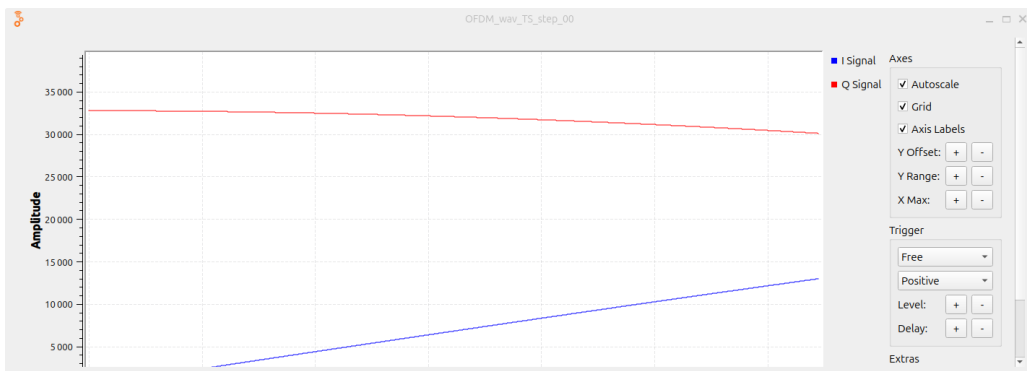


Figure 2: Flowgraph running with the file for $k = 1$: one complete IQ period, $T = 512$ ms

2.3 Description of the Flowgraph Structure

The flowgraph processes the stereo WAV file through the following sequence of blocks.

The WAV File Source reads the stereo file and outputs two independent float streams on separate ports, one for each channel. Each stream passes through a Float to Short block with a scale factor of $65536/2 = 32768$, converting the float values in $[-1, +1]$ to 16-bit signed integers in $[-32768, +32767]$. This scale matches the original amplitude of the generated signals.

A Throttle block on each channel limits the data flow to 2,048,000 samples per second. Without this block, GNU Radio processes data as fast as the processor allows, which would consume all available CPU resources. The Throttle therefore acts as a software clock that paces the simulation to real time.

Each stream then passes through a Short to Float block with scale 1, converting the integers back to float for display purposes while preserving the raw amplitude values. The QT GUI Time Sink then displays both I and Q signals simultaneously on the same plot, with the time axis expressed in microseconds.

3 Question 2 — IQ Interleaving

The two independent I and Q streams are merged into a single 16-bit stream by alternating their samples, producing the sequence $[I_0, Q_0, I_1, Q_1, \dots]$. An Interleave block is inserted between the two Throttle outputs and the display. This block takes the two short integer streams and outputs one stream by taking one sample from each input in turn. With a block size of one sample, it strictly alternates I and Q. The resulting merged stream is converted to float and displayed on the time sink, confirming that the two signals are correctly combined.

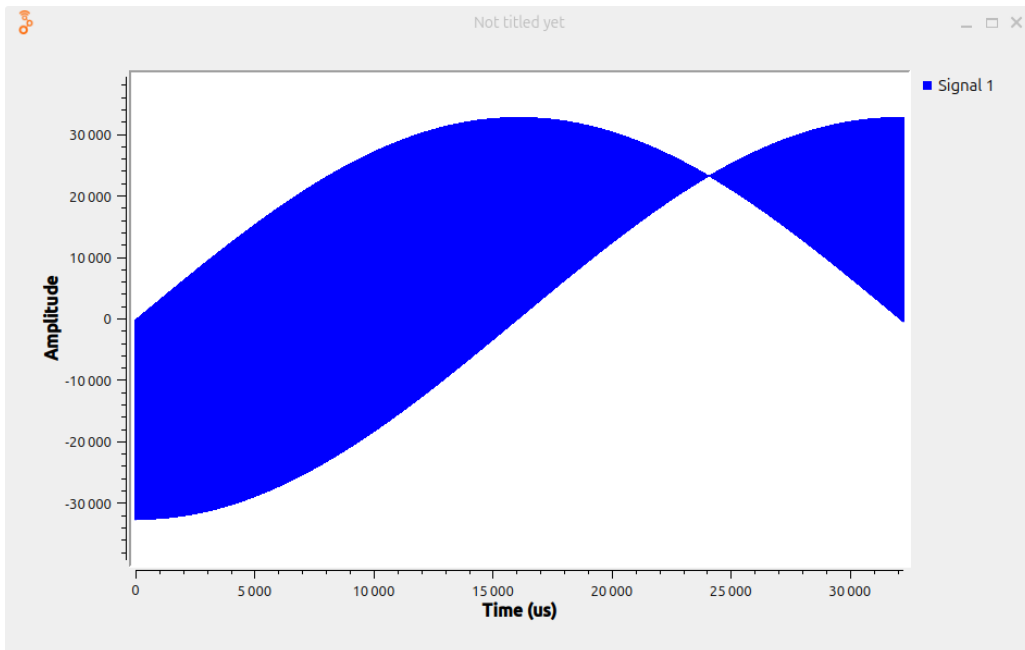


Figure 3: Interleaving flowgraph: the merged IQ stream from the file for $k = 16$ displayed on the time sink

4 Question 3 — QPSK Constellation Encoding

4.1 16-bit to 2-bit Decomposition and Reconstruction

This flowgraph reads the sine file for $k = 16$, implements a complete QPSK encode and decode chain, and plays the reconstructed audio through the audio sink at 48 kHz. The encoding path operates as follows.

First, each 16-bit word is split into two bytes using a custom embedded Python block. The least significant byte is output first, as specified in the lab instructions:

```

1 import numpy as np
2 from gnuradio import gr
3
4 class blk(gr.sync_block):
5     def __init__(self):
6         gr.sync_block.__init__(
7             self,
8             name='my_short_to_char_unpack',
9             in_sig=[np.int16],
10            out_sig=[(np.uint8, 2)]
11        )
12
13    def work(self, input_items, output_items):
14        in_data = input_items[0]
15        out_data = output_items[0]
16        out_data[:, 0] = (in_data & 0xFF).astype(np.uint8)    #
17                       LSB first
18        out_data[:, 1] = ((in_data >> 8) & 0xFF).astype(np.uint8)
19
20    return len(out_data)

```

Listing 2: Python block decomposing each 16-bit word into two bytes, LSB first

The two-byte vectors are then serialised by a Vector to Stream block and passed to a Repack Bits block configured to split each 8-bit byte into four 2-bit symbols, reading from the least significant bit first. One 16-bit word therefore produces eight 2-bit symbols in total. The Constellation Encoder maps each 2-bit symbol to one of four QPSK points using Gray coding, which ensures adjacent symbols differ by exactly one bit. The mapping is: symbol 0 to $-1 - j$, symbol 1 to $-1 + j$, symbol 3 to $+1 + j$, and symbol 2 to $+1 - j$.

The decoding path reverses these operations. After the Constellation Decoder, a Repack Bits block reassembles 2-bit symbols back into bytes, a Stream to Vector block groups them in pairs, and a second custom Python block reconstructs the original 16-bit words:

```

1 import numpy as np
2 from gnuradio import gr
3
4 class blk(gr.sync_block):
5     def __init__(self):
6         gr.sync_block.__init__(
7             self,
8             name='my_short_to_char_pack',

```

```

9         in_sig=[(np.uint8, 2)],
10         out_sig=[np.int16]
11     )
12
13     def work(self, input_items, output_items):
14         input_chars = input_items[0]
15         output       = output_items[0]
16         low_byte     = input_chars[:, 0].astype(np.int16)
17         high_byte    = input_chars[:, 1].astype(np.int16)
18         output[:] = (high_byte << 8) | low_byte
19         return len(output)

```

Listing 3: Python block reconstructing a 16-bit word from two bytes

The use of vectorised NumPy operations avoids Python loops and makes the blocks significantly more efficient. The reconstructed short integers are deinterleaved into two channels, converted to float, and sent to the audio sink.

The constellation obtained has four points at $(\pm 1, \pm j)$. The modulation is therefore **QPSK** (Quadrature Phase Shift Keying), also known as 4-QAM, with a spectral efficiency of 2 bits per symbol.

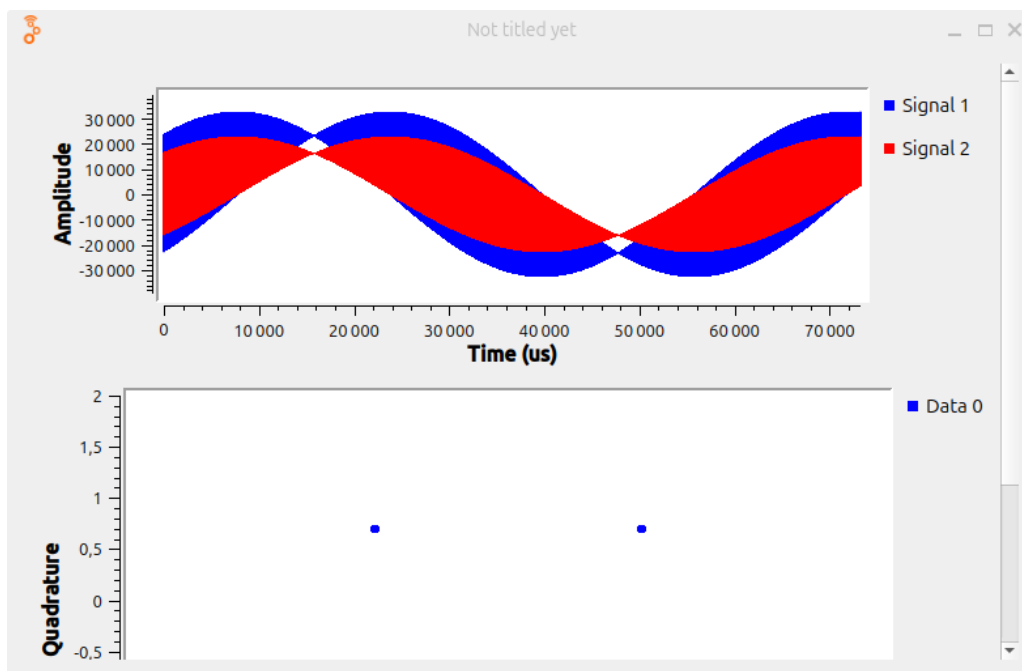


Figure 4: Time sink showing the original interleaved signal and the decoded signal superimposed, confirming lossless reconstruction. Below, the QPSK constellation with four points at the corners of a unit square

4.2 Signal and Constellation Representations

The time sink shows both the original and decoded signals perfectly superimposed throughout the duration of the signal, confirming that the decomposition and reconstruction introduce no errors at any sample. The QPSK constellation shows only two of the four expected points when the input is a pure sinusoid, since a regular sinusoidal signal only

visits two phase values. When tested with a more varied audio signal such as the Vivaldi file, all four points appear.

4.3 Stereo Audio Playback

The third flowgraph plays the Vivaldi audio file in stereo at 48 kHz. The chain passes the audio through the same QPSK encode and decode steps before sending the reconstructed signal to the audio sink. No graphical display is included in this flowgraph, as specified. Successful audio playback confirms that the chain functions correctly with a real audio signal at 48 kHz.

5 Question 4 — $\pi/4$ -DQPSK Modulation

5.1 Theoretical Principle

Phase one: QPSK and DQPSK

QPSK encodes 2 bits per symbol using four absolute phase values: 45, 135, 225, and 315 degrees. The receiver must track and know the absolute carrier phase to decode the signal, which is a significant constraint in practical wireless systems where the channel introduces unknown phase rotations.

DQPSK eliminates this constraint by encoding information in phase *differences* between consecutive symbols rather than in their absolute phases. The receiver compares each received symbol with the previous one:

$$\Delta\phi[n] = \phi[n] - \phi[n-1]$$

This makes the system robust to constant phase offsets introduced by the channel, since a static rotation affects all symbols equally and therefore does not alter the differences.

Phase two: $\pi/4$ -DQPSK

The $\pi/4$ -DQPSK modulation extends DQPSK by alternating between two QPSK constellations that are rotated by $\pi/4$ relative to each other. Each transmitted symbol is encoded as a phase jump chosen from the set:

$$\Delta\phi \in \left\{ +\frac{\pi}{4}, +\frac{3\pi}{4}, -\frac{\pi}{4}, -\frac{3\pi}{4} \right\}$$

The transmitted signal therefore visits 8 states on the IQ plane, alternating between two rotated QPSK grids with each symbol. This brings several important advantages. The signal envelope never passes through zero during transitions, which makes the modulation compatible with non-linear class C power amplifiers without causing spectral regrowth. The differential encoding retains the phase ambiguity immunity of DQPSK. The spectral efficiency remains 2 bits per symbol, identical to standard QPSK. These properties explain the use of this modulation in the DAB digital radio standard, as well as in the TETRA and PDC mobile communication standards.

5.2 Constellation Visualisation

A random source of 1024 two-bit symbols feeds a QPSK constellation encoder, whose output is then processed by the $\pi/4$ -DQPSK modulator. This modulator is implemented as an embedded Python block that accumulates phase jumps while maintaining memory of the previous phase between successive processing calls:

```

1 import numpy as np
2 from gnuradio import gr
3
4 class blk(gr.sync_block):
5     def __init__(self):
6         gr.sync_block.__init__(
7             self,
8             name='pi4_dqpsk_mod',
9             in_sig=[np.uint8],
10            out_sig=[np.complex64]
11        )
12        self.phase = 0.0 # phase memory between blocks
13
14    def work(self, input_items, output_items):
15        in_data = input_items[0]
16        phases_jumps = np.array([np.pi/4, 3*np.pi/4, -np.pi/4,
17                                -3*np.pi/4])
18        current_jumps = phases_jumps[in_data]
19        accumulated_phases = np.cumsum(current_jumps) + self.
20            phase
21        self.phase = accumulated_phases[-1] % (2*np.pi)
22        output_items[0][:] = np.exp(1j * accumulated_phases)
23        return len(output_items[0])

```

Listing 4: Python block implementing the $\pi/4$ -DQPSK phase accumulation modulator

Three constellations are overlaid on the same display. The blue data set (Data 0) shows the QPSK output from the constellation encoder, which forms four points at the corners of a square. However, since the QPSK points coincide exactly with the positions of the other two constellations, they are not visible when all three are displayed simultaneously — they are hidden beneath the red and green points. The figure below shows all three overlaid, followed by the blue constellation displayed alone by hiding the other two in the legend.

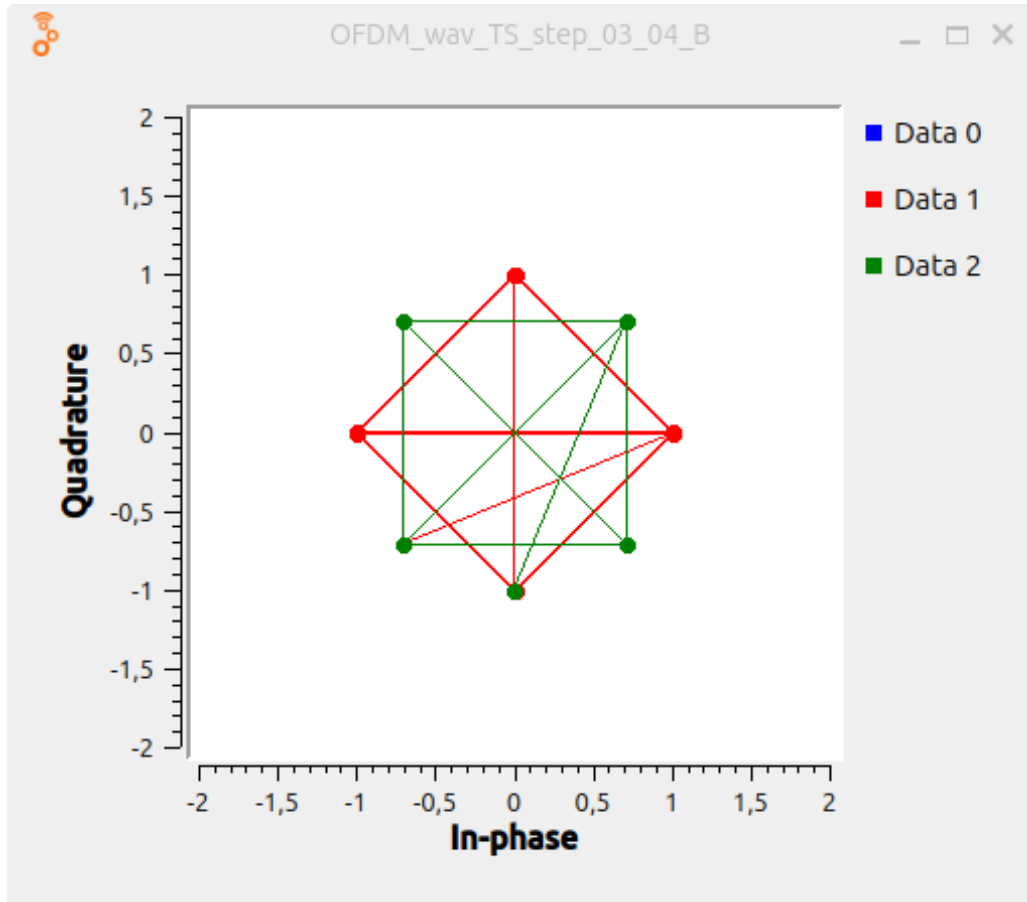


Figure 5: Three constellations overlaid on the same display: QPSK in blue (Data 0), QPSK reference in red (Data 1), and $\pi/4$ -QPSK reference in green (Data 2). The blue points are hidden beneath the others

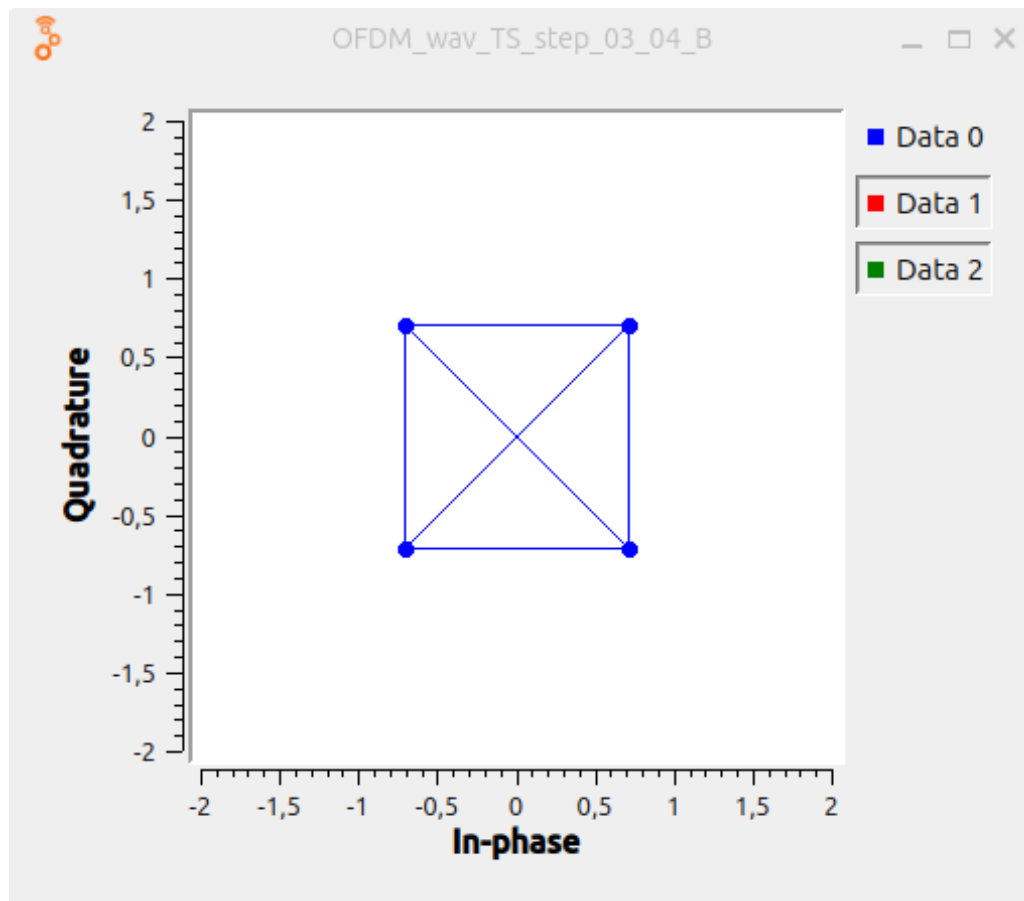


Figure 6: The same display with only Data 0 visible: the four QPSK points at the corners of a square are now clearly shown in blue

5.3 Fixed Sequence and Frame Synchronisation

A fixed sequence of four symbols repeated continuously feeds the modulation chain. A Stream to Tagged Stream block inserts a tag every four symbols, carrying the label indicating the packet length. The time sink is configured to trigger on this tag, so the display always starts at the beginning of a new frame. This mechanism achieves frame synchronisation without a dedicated detector: the tag propagates through the signal path and can be used downstream by the demodulator to align its processing without needing to search for a frame start marker.

The time sink shows the phase angle of the modulated signal in units of $\pi/4$ alongside a QPSK reference, producing the characteristic sawtooth pattern for the fixed input sequence.

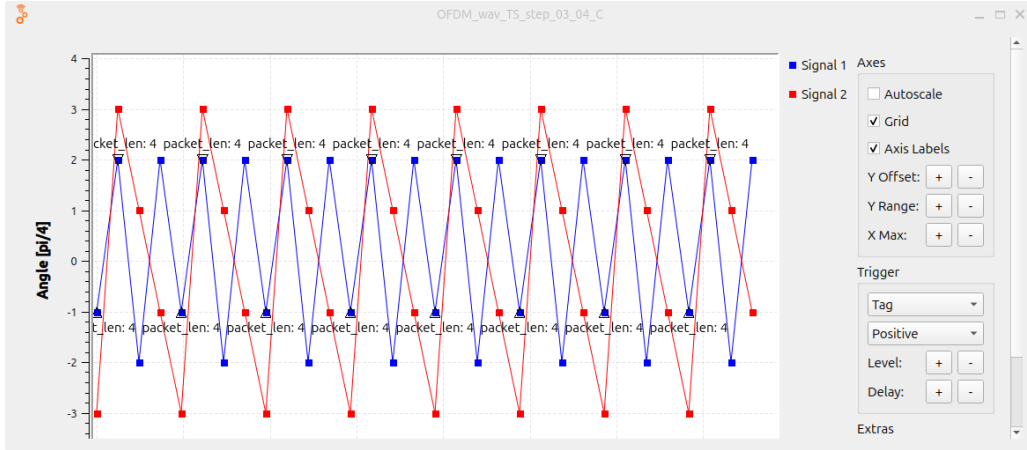


Figure 7: Phase angles in units of $\pi/4$ over time with frame tags marking each boundary, confirming frame synchronisation via the tag mechanism

5.4 Noisy Channel

A Channel Model block is added after the modulator to introduce additive white Gaussian noise. The taps parameter is set to 1 so that no phase offset is introduced at this stage. The noise voltage is controlled in real time by a slider. Both a time sink and a constellation sink show the degradation of the signal as the noise level increases. The eight states of the $\pi/4$ -DQPSK constellation spread progressively until they overlap and demodulation becomes unreliable.

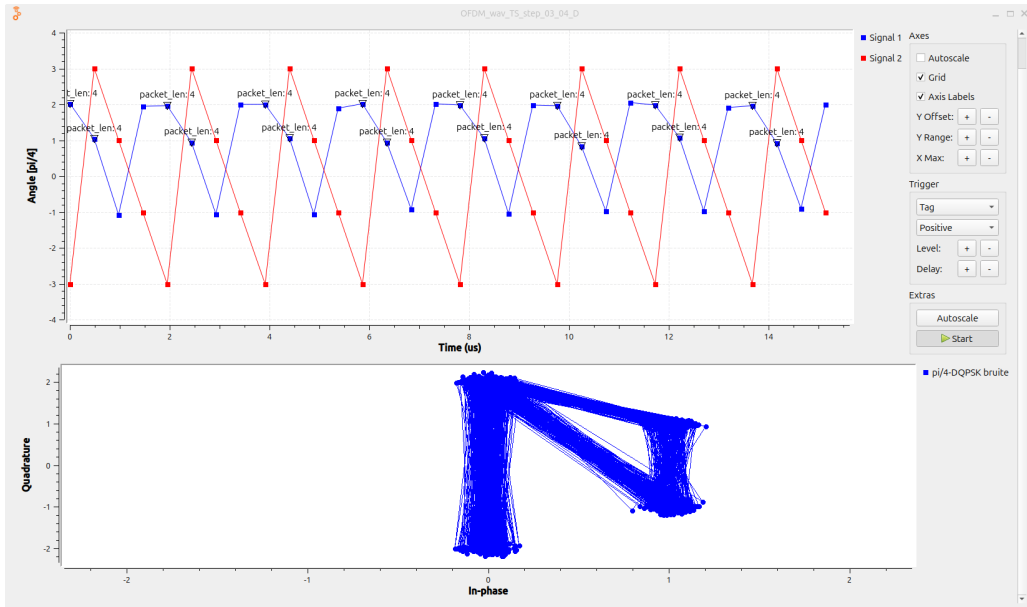


Figure 8: Noisy $\pi/4$ -DQPSK signal: time sink with frame tags and constellation sink showing the spreading effect of the noise

5.5 Phase Offset and Delay Compensation

Two modifications are introduced. First, a constant phase rotation of $\pi/10$ is applied to the channel by setting the taps parameter to $\cos(\pi/10) + j \sin(\pi/10)$. This multiplies all

received symbols by $e^{j\pi/10}$, rotating the entire constellation by 18 degrees.

Second, the Channel Model block introduces an internal processing delay of approximately three samples. Without correction, the frame tag transmitted through the tagged stream path arrives three samples earlier than the data that has passed through the channel, causing a temporal misalignment. A Delay block of three samples is inserted on the tagged stream path to compensate this offset. The tag is then re-aligned with the delayed data by multiplying the two paths together, allowing the time sink to trigger correctly without a frame detector.

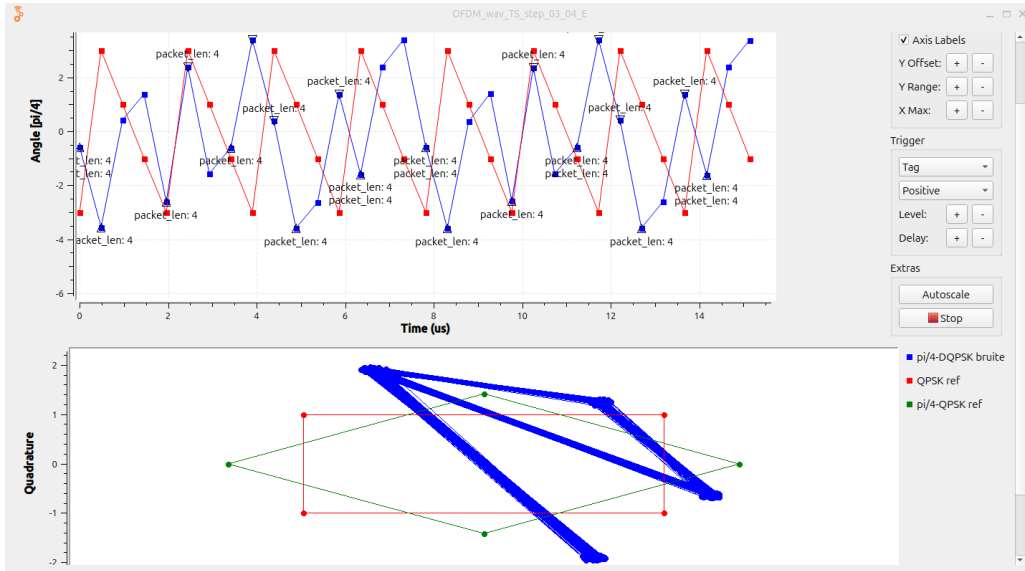


Figure 9: Constellation showing the $\pi/10$ phase offset: the received signal cluster (blue) is rotated 18 degrees relative to the clean QPSK reference (red) and $\pi/4$ -QPSK reference (green)

5.6 DQPSK and QPSK Recovery

This flowgraph uses a random source and a channel with pure AWGN and no phase offset. A differential demodulator Python block computes, for each received sample $s[n]$, the phase difference $\Delta\phi[n] = \arg(s[n] \cdot s^*[n-1])$ and maps it to the nearest allowed phase jump:

```

1 import numpy as np
2 from gnuradio import gr
3
4 class blk(gr.sync_block):
5     def __init__(self):
6         gr.sync_block.__init__(
7             self,
8             name='pi4_dqpsk_demod',
9             in_sig=[np.complex64],
10            out_sig=[np.uint8]
11        )
12        self.prev_sample = complex(1, 0)
13
14    def work(self, input_items, output_items):
```

```

15     in_data = input_items[0]
16     out_data = output_items[0]
17     N = len(in_data)
18     samples = np.concatenate([[self.prev_sample], in_data])
19     delta_phi = np.angle(samples[1:] * np.conj(samples[:-1])
20                           )
21     self.prev_sample = in_data[-1]
22     possible_phases = np.array([np.pi/4, 3*np.pi/4, -np.pi
23                               /4, -3*np.pi/4])
24     symbol_map = np.array([0, 1, 2, 3])
25     for i in range(N):
26         diffs = np.abs(np.angle(np.exp(1j*(delta_phi[i] -
27                                     possible_phases))))
28         out_data[i] = symbol_map[np.argmin(diffs)]
29     return N

```

Listing 5: Python block implementing differential $\pi/4$ -DQPSK demodulation

The decoded symbols are re-encoded as QPSK to allow visualisation of the recovered constellation. The constellation sink shows four data sets simultaneously: the noisy $\pi/4$ -DQPSK signal in blue, the clean QPSK reference in red, the clean $\pi/4$ -QPSK reference in green, and the decoded QPSK clusters in black. The four black clusters positioned at the expected QPSK points confirm that the differential detector correctly recovers the original symbols despite the noise.

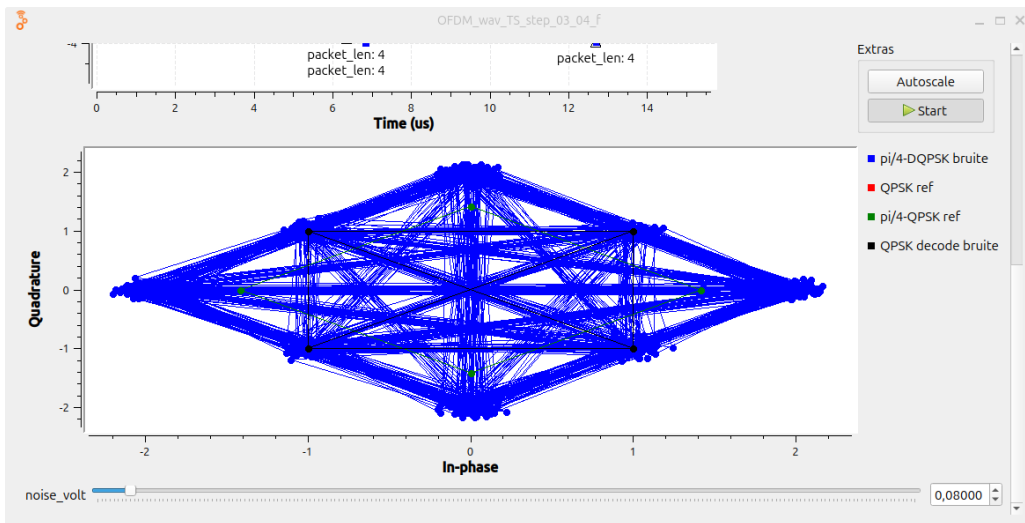


Figure 10: Constellation with four data sets: $\pi/4$ -DQPSK signal in blue, QPSK reference in red, $\pi/4$ -QPSK reference in green, decoded QPSK in black

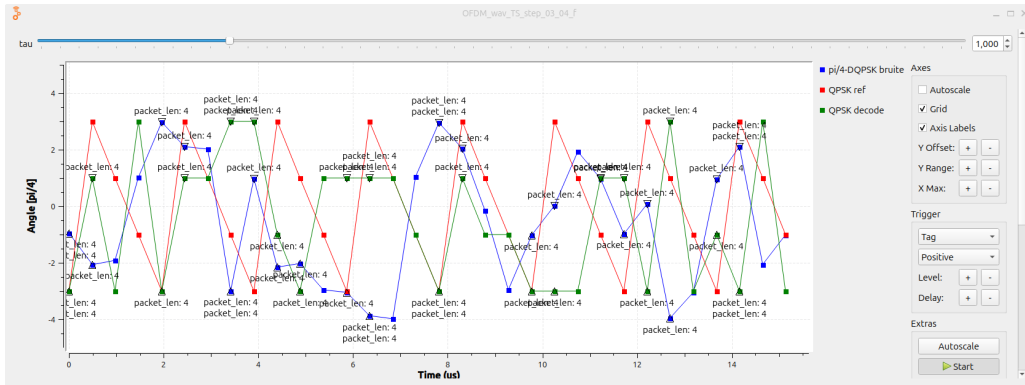


Figure 11: Time sink showing three phase signals: $\pi/4$ -DQPSK in blue, QPSK reference in red, decoded QPSK in green

6 Question 5 — Complete Audio Transmission Chain

The complete chain processes the short Vivaldi audio file, which is a stereo recording at 48 kHz, through $\pi/4$ -DQPSK modulation and demodulation using two dedicated hierarchical blocks.

The first hierarchical block, which handles modulation, groups the incoming QPSK symbols into vectors of five elements: one pilot symbol equal to $(1 + j)/\sqrt{2}$ followed by four data symbols. It then applies cumulative differential multiplication to each vector, producing the $\pi/4$ -DQPSK signal. The pilot symbol serves as the phase reference for the differential encoding. The second hierarchical block performs the inverse operation using conjugate-based differential detection and removes the pilot symbol from the output.

The Throttle block is set to 96,000 samples per second, which is twice the audio rate of 48,000. This accounts for the fact that the stereo audio has been interleaved into a single stream, doubling the effective data rate before the throttle. The decoded output is sent both to the audio sink for direct listening and to a WAV file sink where it is saved as a received file for verification by playback.

The time sink, configured with 64 display points at the full sampling rate of 2.048 MHz, shows the $\pi/4$ -DQPSK modulated signal. The constellation sink shows the QPSK symbols after demodulation.

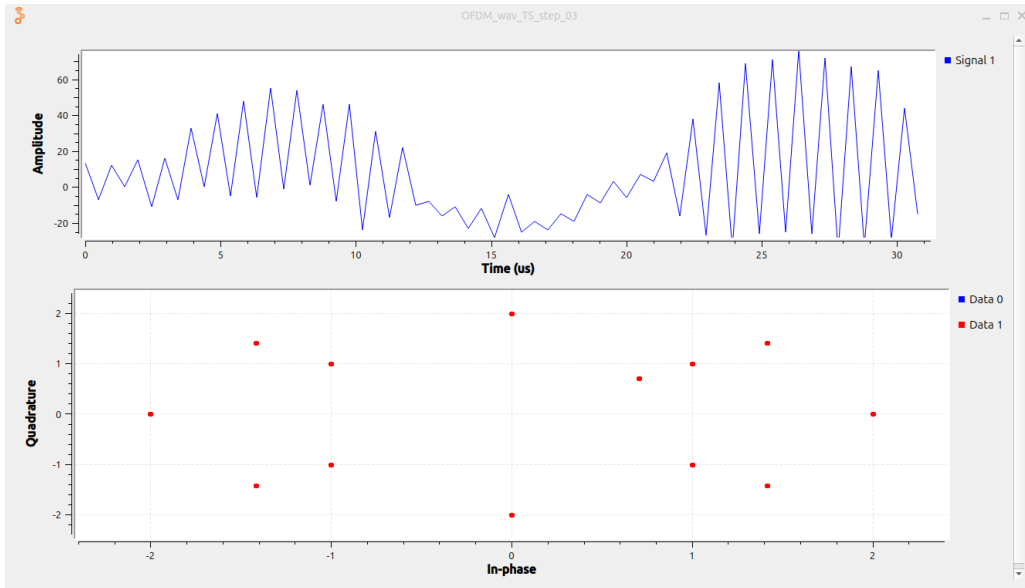


Figure 12: Complete flowgraph for the audio chain: the $\pi/4$ -DQPSK signal is visible on the time sink and the demodulated QPSK symbols appear on the constellation sink

7 Question 6 — Noise Limit and Phase Ambiguity

7.1 Noise Tolerance Limit

Using the noisy channel flowgraph from Question 4f, the noise voltage slider is progressively increased from zero until the eight signal states in the constellation begin to overlap and demodulation errors become frequent. This threshold is reached at approximately 0.5 V of noise voltage, below which the eight states remain separable and the chain operates correctly.

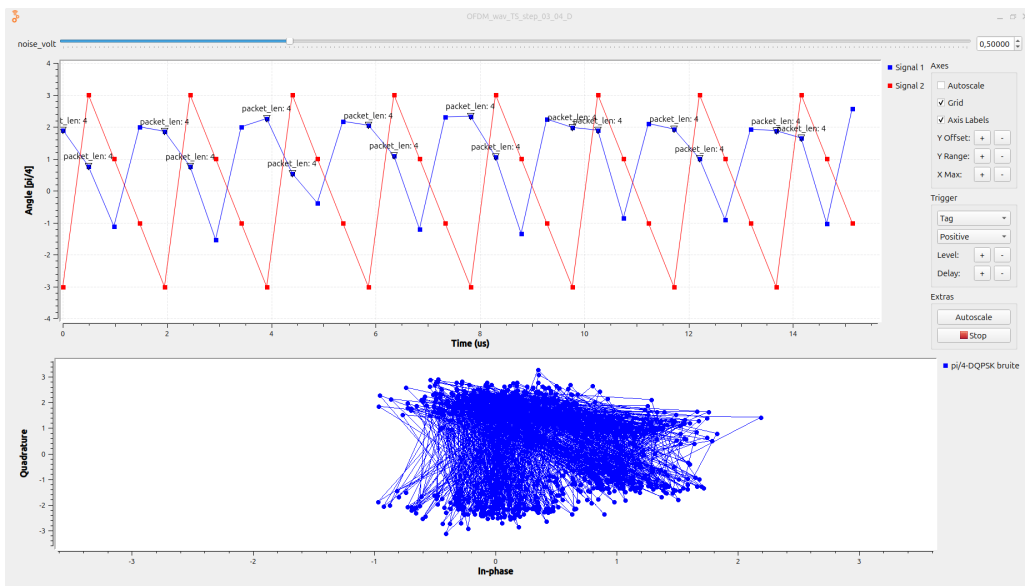


Figure 13: Constellation near the noise tolerance limit: the eight states begin to overlap, indicating the boundary of reliable demodulation

7.2 Phase Ambiguity Immunity

Using the phase offset flowgraph from Question 4g with a favourable noise level, a phase rotation of $\pi/10$ is applied through the channel. The raw $\pi/4$ -DQPSK constellation rotates by 18 degrees as expected. However, the decoded QPSK clusters obtained through differential detection remain aligned with the clean reference, confirming that a constant phase offset introduced by the channel does not affect the decoded symbols. Only the phase differences between consecutive symbols carry information, so a static rotation cancels out in the subtraction.

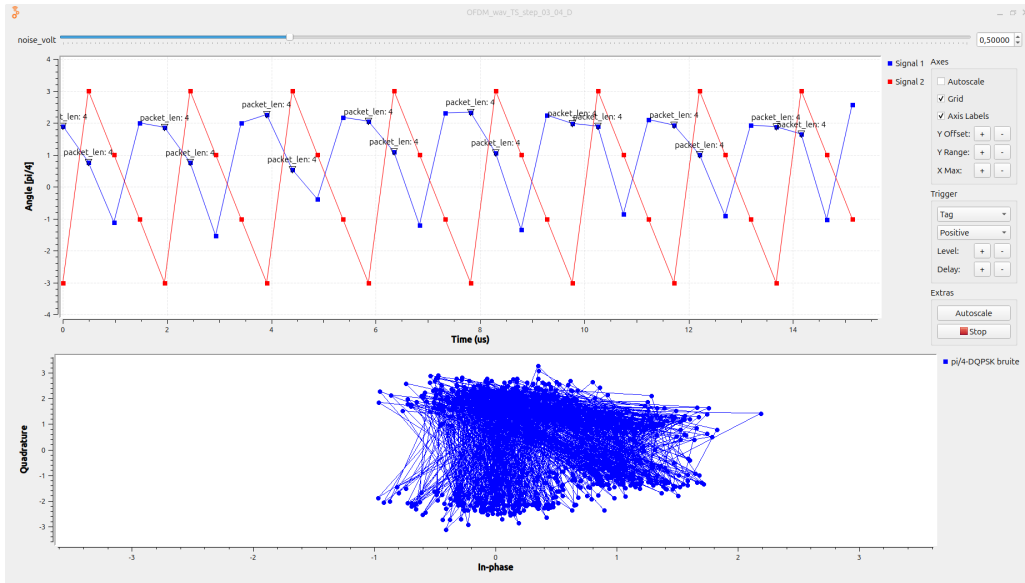


Figure 14: At a favourable noise level with a $\pi/10$ phase rotation: the decoded QPSK clusters remain correctly positioned despite the rotation of the received signal, confirming phase ambiguity immunity

8 Question 7 — OFDM with 64 Subcarriers

An OFDM system with 64 subcarriers is implemented using the provided model. A cosine signal at 1000 Hz and a sample rate of 32 kHz is used as the test input to validate the chain. The signal is grouped into blocks of 64 samples by a Stream to Vector block, then transformed to the frequency domain by a forward FFT of size 64. Each of the 64 frequency-domain coefficients is processed by the IQ to $\pi/4$ -DQPSK hierarchical block operating at 2.048 MHz, which modulates each subcarrier independently. The reverse path applies the $\pi/4$ -DQPSK to IQ hierarchical block, followed by a forward FFT of size 64 that reconstructs the frequency-domain representation, and finally a Multiply Const block that normalises the amplitude by a factor of $1/64$ to compensate for the FFT scaling. The difference between the transmitted and received signals is computed by a Subtract block and its magnitude is displayed on the time sink.

The 64 subcarriers are mutually orthogonal by construction of the FFT, which prevents inter-carrier interference and allows each subcarrier to carry an independent modulated data stream. This frequency diversity also provides inherent robustness against selective fading. The same principle underlies the DAB standard, which uses 1536 subcarriers along with a cyclic prefix guard interval to handle multipath propagation.

A cosine test signal is used here rather than the Vivaldi audio file, since the audio file is sampled at 48 kHz while the flowgraph operates at 32 kHz. The test signal at 1 kHz is sufficient to validate the OFDM modulation and demodulation chain.

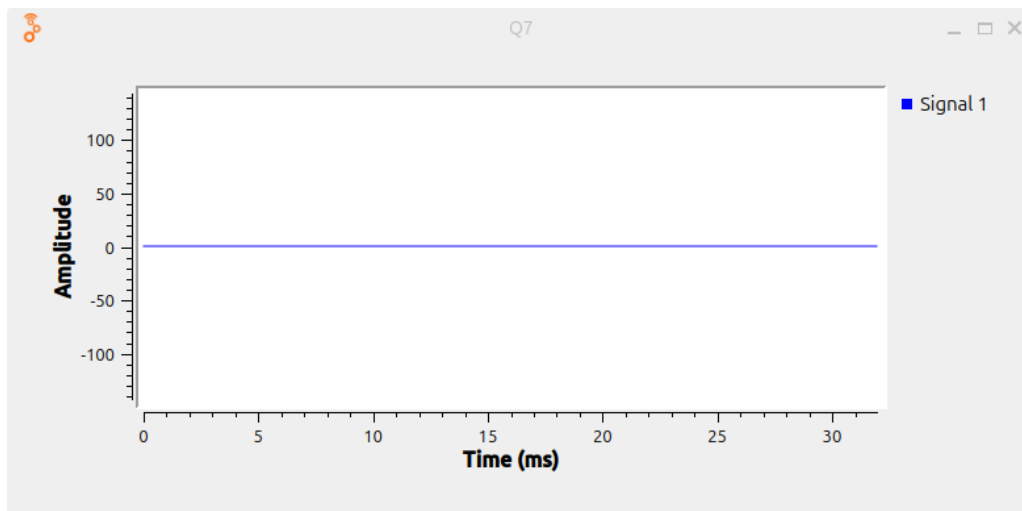


Figure 15: OFDM flowgraph with 64 subcarriers: the residual error magnitude between transmitted and received signals is displayed on the time sink

9 Conclusion

This lab implemented a complete physical layer transmission chain in GNU Radio Companion, following the architecture of the DAB digital radio standard. Each stage was built and validated before moving to the next:

Question	Flowgraph	Outcome
Q1	Generation script and first flowgraph	IQ generation and period verified
Q2	Interleaving flowgraph	IQ interleaving validated
Q3a–b	QPSK chain flowgraph	Lossless QPSK encoding confirmed
Q3c	Vivaldi playback flowgraph	Stereo audio playback successful
Q4a	—	QPSK, DQPSK and $\pi/4$ -DQPSK principles explained
Q4b–d	Constellation display flowgraph	Three constellations displayed
Q4e	Fixed sequence flowgraph	Frame synchronisation via stream tags
Q4f	Noisy channel flowgraph	AWGN effect quantified
Q4g	Phase offset flowgraph	Delay bias compensated, phase rotation visible
Q4h	Demodulation flowgraph	Differential demodulation validated
Q5	Complete audio chain	Full $\pi/4$ -DQPSK audio chain functional
Q6	Noise and phase tests	Noise limit at approximately 0.5 V, phase ambiguity resolved
Q7	OFDM flowgraph	64-subcarrier OFDM chain validated

The $\pi/4$ -DQPSK modulation proved particularly well suited to broadcast applications. Its differential encoding eliminates the need for carrier phase synchronisation at the receiver. The alternating constellation grids ensure the signal envelope never crosses zero, preserving compatibility with non-linear power amplifiers. Combined with OFDM multiplexing, these properties constitute the physical layer of the DAB standard, which has

been broadcasting digital radio across Europe since 1995.